



COS 132 – Imperative Programming

Foundation Test #3 Memo

Worked Solutions

Module:	COS 132 – Imperative Programming
Assessment:	Foundations Test #3 (ST2 Programme)
Set by:	Swift Tuition
Total Marks:	25

Programme note. This material is for Day 1 of the 7-day Swift Tuition COS 132 ST2 programme. The premium course is available at www.swifttuition.com/tuks/cos132. Learners watching before ST2 2026 may use coupon code **200COS132** at checkout for R200 off the full price. All three foundation tests are included; one test can be taken now and the remaining tests anytime after ST2. Where units are mentioned, they refer to Swift Tuition’s learning-unit split of the module’s lecture content, covering lectures 1–16.

1 Overview

Good day, good people. Below is the worked solution for Foundations Test #3 — the hardest of the foundations bank, calibrated to exam-realistic difficulty. Read the reasoning, not just the answers.

Total marks: 25

Question 1

8 marks

(a) True or False

i. True. [1 mark]

This is testing whether you know the connection between `char` and ASCII. Every character is stored as a small integer (its ASCII code), and you can convert in either direction with a `static_cast`.

65 happens to be the ASCII code for capital A. 66 is B, 67 is C, and so on through 90 for Z. Lowercase starts at 97 (a). Spotting $65 \rightarrow 'A'$ is something to commit to memory.

`static_cast<char>(65)` converts the `int 65` into a `char`, giving you `'A'`.

ii. False. [1 mark]

C++ compares `int` and `char` happily. The `char` gets implicitly promoted to its integer value, and then the comparison runs as an `int == int`.

So this works:

```
char c = 'A';
if (c == 65) {
    cout << "Yes, A is 65 in ASCII";
}
```

The compiler quietly converts `c` to its integer code (65), then compares `65 == 65`, which is `true`.

This implicit promotion is convenient for ASCII tricks (like checking `c >= '0' && c <= '9'` to test if a `char` is a digit), but it means C++ is more permissive than some other languages here.

iii. True. [1 mark]

`const int x` says "the parameter `x` cannot be modified inside this function". You can read `x` as much as you like, but any attempt to change it (`x = 5`, `x++`, `x += 1`) is a compile error.

```
void f(const int x) {
    cout << x;           // OK: reading
    x = 10;              // ERROR: const
    x++;                 // ERROR: const
}
```

This is a common pattern when you want to make it crystal clear that a parameter is input-only.

iv. True. [1 mark]

C++ lets you declare and initialise multiple variables of the same type in one statement, separated by commas:

```
int x = 5, y = x + 2;
```

This declares two `ints`. `x` is initialised to 5, and `y` is initialised to `x + 2 = 7`. Both happen in one statement.

The rules:

- Both variables must have the same base type (both `int` here).
- Each one can have its own initialiser.
- The initialiser for `y` can refer to `x`, because `x` is declared first (left-to-right within the statement).

(b) Multiple choice

i. A. 2.5. [1 mark]

This is the integer-division trap, hiding inside a more elaborate expression. Trace the order of operations:

`a / b` — both `int`, so integer division. $5 / 2 = 2$.

$2 + 0.5$ — now we have `int` + `double`. The `int` promotes to `double`, so this is $2.0 + 0.5 = 2.5$.

Final assignment: `result = 2.5`.

The half (0.5 from $5/2$) was lost in step one because both operands were integers. The 0.5 literal that comes later doesn't recover it.

If you picked C (3.5), you assumed $5 / 2$ produces 2.5 . It doesn't, because both `a` and `b` are `int`. The fractional part is discarded before the addition.

ii. B. 5. [1 mark]

Trace the loop with `i = 1`:

- Iteration 1: $1 < 5$ true, `i` becomes 2.
- Iteration 2: $2 < 5$ true, `i` becomes 3.
- Iteration 3: $3 < 5$ true, `i` becomes 4.
- Iteration 4: $4 < 5$ true, `i` becomes 5.
- Check: $5 < 5$ false. Exit the loop.

The loop body increments `i` *until* $i \geq 5$, but the value is checked again *after* the increment. So the moment `i` becomes 5, the next check fails and the loop exits — but `i` is now 5, not 4.

Common mistake: thinking the loop "stops at 4" because $4 < 5$ was the last *true* check. It doesn't — the increment ran one more time after that, taking `i` to 5.

iii. B. Integer division before conversion. [1 mark]

Consider what happens when this function is called as `divide(7, 2)`:

1. `a = 7, b = 2`. Both are `int`.
2. `a / b` is integer division. The result is 3 (an `int`), not 3.5 .
3. The function's return type is `double`, so the `int` 3 is converted to `double` 3.0 on its way out.
4. The caller receives 3.0 . The fractional part was already lost in step 2.

The fix would be to cast at least one operand to `double` *before* dividing:

```
return static_cast<double>(a) / b;
```

Now the division is floating-point from the start, and you get 3.5 .

iv. B. Prevents multiple inclusion in a single translation unit. [1 mark]

The `#ifndef ... #define ... #endif` pattern is called an **include guard**. It works like this:

```
#ifndef Geometry_h
#define Geometry_h

// ... contents of the header ...

#endif
```

The first time the preprocessor sees this header, the macro `Geometry_h` is undefined, so the `#ifndef` is true. It defines the macro and processes the header contents.

The second time the same header is `#included` (either directly or transitively through some other header), the macro is now defined, so the `#ifndef` is false. The whole block is skipped.

Without an include guard, including the same header twice would re-declare every type and function inside it, which is a compile error.

Modern alternatives like `#pragma` once do the same job in one line, but include guards are the textbook standard and what COS 132 expects.

Question 2

6 marks

(a) Output: 5 6 8 [3 marks]

This is a multi-step overload-resolution question. There are two overloads:

- `int compute(int a, int b = 3)` — takes one or two `ints`, returns the sum.
- `double compute(double a, double b)` — takes two `doubles`, returns the product.

Now resolve each call:

`compute(2)` — one `int` argument. The `int` overload matches (with `b` defaulting to 3). The `double` overload doesn't match because it requires two arguments. Result: $2 + 3 = 5$.

`compute(2, 4)` — two `int` arguments. The `int` overload matches exactly. The `double` overload would require both arguments to be `double`, and an exact match wins over implicit conversion. Result: $2 + 4 = 6$.

`compute(2.0, 4.0)` — two `double` arguments. Now the `double` overload matches exactly. Result: $2.0 * 4.0 = 8$. (Default `cout` formatting drops the `.0` for whole-number doubles, so this prints as 8, not 8.0.)

Output: 5 6 8.

Common mistake: picking the `double` overload for `compute(2)` because of the default argument. The compiler resolves overloads on the **arguments you actually pass**, before considering defaults. Since `compute(2)` provides one `int`, the `int` overload is the only candidate.

(b) Output: 20 [3 marks]

This is a nested loop where the inner loop's bound depends on the outer counter. Trace it carefully:

outer <code>i</code>	inner runs	values added	running total
1	<code>j = 1</code>	1	1
2	<code>j = 1, 2</code>	1, 2	$1 + 1 + 2 = 4$
3	<code>j = 1, 2, 3</code>	1, 2, 3	$4 + 1 + 2 + 3 = 10$
4	<code>j = 1, 2, 3, 4</code>	1, 2, 3, 4	$10 + 1 + 2 + 3 + 4 = 20$

The pattern: for each outer iteration, the inner loop sums 1 through `i`. The grand total adds those triangular numbers: $1 + 3 + 6 + 10 = 20$.

If you got 10, you only counted the inner sums for `i = 1..3` and missed the `i = 4` row.
If you got 30, you probably overshot one iteration in either loop. Re-trace with care.

Question 3

5 marks

(a) areaOfCircle implementation [2 marks]

```
#include "Constants.h"

// ...

double areaOfCircle(double radius) {
    return PI * radius * radius;
}
```

How to think about it

The formula is $\pi \times r^2$. In C++, you have three pieces:

- **PI** is the constant defined in `Constants.h`.
- **radius** is the parameter.
- Squaring the radius means multiplying it by itself: `radius * radius`. (You could use `pow(radius, 2)` from `<cmath>`, but the simple multiplication is faster and clearer for an exponent of 2.)

You also need to make sure the file knows where **PI** lives. Since **PI** is declared in `Constants.h`, you must `#include "Constants.h"` at the top of `Geometry.cpp`. Without it, the compiler complains that **PI** is undefined.

Marks breakdown

- 1 mark for the formula `PI * radius * radius` (or an equivalent like `PI * pow(radius, 2)`).
- 1 mark for adding `#include "Constants.h"` (or noting that this addition is needed).

Common pitfalls

- **Hard-coding 3.14159 instead of using PI.** The spec is explicit that **PI** is defined in `Constants.h`. Hard-coding earns only the formula mark, not the include mark.
- **Forgetting the include.** The function won't compile without it. If your answer mentions the include — even just in a comment — you get the mark.

(b)(i) Makefile entry for Geometry.o [2 marks]

```
Geometry.o: Geometry.cpp Geometry.h Constants.h
<TAB>g++ -c Geometry.cpp
```

The `<TAB>` indicates a single tab character at the start of the recipe line. This is **mandatory** — `make` refuses to run if you use spaces instead of a tab there. It's one of the most famously fiddly rules in `make`.

How to think about it

A makefile entry has three parts:

1. **The target** — what you're trying to build (`Geometry.o`, the object file).
2. **The dependencies** — what files the target depends on. If any of these change, the target needs to be rebuilt. For an object file, the dependencies are: the `.cpp` file itself, plus every header it includes. So `Geometry.cpp`, `Geometry.h` (because `Geometry.cpp` includes it) and `Constants.h` (because we just added that include in part (a)).
3. **The recipe** — the command to run. For object files, that's `g++ -c filename.cpp`. The `-c` flag means "compile only, don't link" — produce `Geometry.o` and stop.

Marks breakdown

- 1 mark for the target line listing the right dependencies (`Geometry.cpp`, plus the headers).
- 1 mark for the recipe line with `g++ -c Geometry.cpp` and the tab indicated.

Common pitfalls

- **Forgetting the tab.** Even if your makefile *looks* right, `make` will reject it with the cryptic error `*** missing separator. Stop.` if there's no tab.
- **Forgetting `-c`.** Without it, `g++` tries to link as well, which fails because `main` isn't in `Geometry.cpp`.
- **Listing only `Geometry.cpp` as a dependency.** If `Geometry.h` changes, your makefile won't know to rebuild `Geometry.o`. Always include the headers your `.cpp` file uses.

(b)(ii) Linker entry for Main [1 mark]

```
Main: main.o Geometry.o
<TAB>g++ -o Main main.o Geometry.o
```

How to think about it

This is the linker step. You're combining the already-built object files (`main.o` and `Geometry.o`) into a single executable. The `-o Main` flag tells `g++` to call the output file `Main`. Without it, you'd get the default name `a.out`.

Common pitfalls

- **Missing `-o Main`.** You'd build a working executable, but it would be called `a.out`, which doesn't match what the question asked for.
 - **Listing source files instead of object files.** This entry expects `.o` files, not `.cpp` files. `g++` would still produce a working program, but you'd be re-doing the compile step that you already specified separately, defeating the purpose of having both entries.
-

Question 4

6 marks

(a) printGrid implementation [4 marks]

```
void printGrid(int rows, int cols) {
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            cout << "*";
        }
        cout << endl;
    }
}
```

How to think about it

The Logo procedure does two nested things:

- **Outer:** repeat :ROWS times, where each repetition is a row.
- **Inner:** repeat :COLS times, printing one * per repetition without a newline.
- After the inner loop finishes, print an empty line (which produces a newline) so the next row starts on a fresh line.

In C++:

- The outer REPEAT :ROWS becomes a for loop running rows times.
- The inner REPEAT :COLS becomes a nested for loop running cols times.
- PRINT1 "*" (no newline) becomes cout << "*"; — note the absence of endl.
- PRINT "" (just a newline) becomes cout << endl; — placed *after* the inner loop, so it runs once per row.

The placement of the endl is the most important detail. If you put it inside the inner loop, every star ends up on its own line. If you forget it entirely, all the stars run together on a single line.

Marks breakdown

- 1 mark: function signature void printGrid(int rows, int cols).
- 1 mark: outer loop runs rows times.
- 1 mark: inner loop runs cols times, printing a star without a newline.
- 1 mark: a newline after each row (and not inside the inner loop).

Other valid answers

Using '*' (char literal) instead of "*" (string literal): equivalent. Counting from 1 to rows (inclusive) instead of 0 to rows - 1: equivalent, as long as the loop runs the right number of times. A while loop variant: fine.

Common pitfalls

- **Newline in the wrong place.** The single most common mistake. cout << endl; belongs *between* the inner-loop end and the next outer iteration — outside the inner loop's braces.
- **Off-by-one.** r <= rows with r starting at 0 runs the outer loop rows + 1 times. Pick a consistent convention (0 to n-1 with <, or 1 to n with <=) and stick with it.

(b) Modified version (only odd rows) [2 marks]

The cleanest answer:

```
void printGrid(int rows, int cols) {
    for (int r = 1; r <= rows; r += 2) {
        for (int c = 0; c < cols; c++) {
            cout << "*";
        }
    }
}
```



```

    }
    cout << endl;
}
}

```

How to think about it

The spec says "rows whose row index, starting at 1, is odd". So you want rows 1, 3, 5, 7, ... — every second one, starting from 1.

The most direct way is to make the loop step by 2 instead of 1. Start `r` at 1, increment by `r += 2` each time, and the loop visits exactly the odd indices. No `if` needed.

Other valid answers

A version that loops over every row but uses an `if` to filter is also fully correct:

```

void printGrid(int rows, int cols) {
    for (int r = 1; r <= rows; r++) {
        if (r % 2 == 1) {
            for (int c = 0; c < cols; c++) {
                cout << "*";
            }
            cout << endl;
        }
    }
}

```

Same output, full marks. The step-by-2 version is just a touch more idiomatic.

A `continue`-based version is also fine:

```

for (int r = 1; r <= rows; r++) {
    if (r % 2 == 0) continue;
    // ... print the row
}

```

Marks breakdown

- 1 mark: the modified loop produces only odd-indexed rows.
- 1 mark: the modified function still produces correct output for the rows it does print.

Common pitfalls

- **Printing rows 2, 4, 6 instead of 1, 3, 5.** The spec was explicit: indices start at 1. If you're using a 0-indexed loop, the *odd indices in that frame* are the *even row numbers* in the spec's frame. Either re-index from 1, or flip the condition.
- **Forgetting the inner-loop body.** The modification is to the row-selection logic, not the row-printing logic. The inner loop and the trailing `endl` should still be there for every row that is printed.

After this test

This was the hardest of the foundations bank, calibrated to exam-realistic ST1 difficulty. Roughly:

- **22–25 marks:** ST1 territory is solid. You're ready for Unit G.
- **17–21 marks:** mostly there, but the harder corners (overload resolution, makefile syntax, nested loops) need tightening. Replay the relevant lecture videos and try one of the easier foundations papers as a confidence rebuild.
- **12–16 marks:** real gaps in specific spots. Identify which questions caught you, go back to those topics in Units

C–F, and retry this paper in a few days.

- **Below 12:** retake Foundations #1 or #2 first; come back to this one once you're scoring above 18 there.

Logo coverage

This test contained one Logo→C++ conversion (Q4) — nested REPEAT loops with PRINT1 (no newline) and PRINT "" (newline only) primitives, plus a follow-up modification asking for filtered iteration. Per the COS 132 lecturer's confirmation that ST2 can ask Logo→C++ conversions including loops, this is the harder end of what you might see at the foundations level, and it previews the kind of multi-loop reasoning Unit G's Logo→C++ questions will demand.